

JS Back-End Regular Exam – Space Mission Control

1. Exam Rules:

1. You have **6 hours**.
2. When you are ready, delete the **node_modules** folder, make sure all dependencies are listed in the **package.json** file and submit your archiving project.
3. You must use the provided HTML & CSS resources.
4. You may **add attributes** (such as **class** and **dataset**), but it is forbidden to **change existing** attributes (such as **class** and **id**).
5. You may **change "href"** attributes on links and add/change the **method** and **action** attributes of HTML Forms.
6. Use **Express.js** as a back-end framework.
7. Use **PostgreSQL** as a database with **Prisma** ORM.
8. Use **express-handlebars** as a view engine.
9. Use **bcrypt** for hashing the password.
10. Use **Zod** for input validation.
11. It is **forbidden** to use **React**, **Vue**, **Angular**, etc.
12. All applications will be **examined** as follows:
 - The needed **dependencies** are installed with the command **"npm install"**
 - The **database client** is generated with the command **"npx prisma generate"**
 - The database tables are created with the command **"npx prisma db push"**
 - The application is **started** with the command **"npm run start"**
 - The DATABASE_URL in the **.env** file will be changed to Mission the local database configuration.
13. The examiners are **NOT** allowed to **change** anything in your code and will **NOT** use any **instruments outside** of the course scope.
14. You decide how to name your links.

IMPORTANT:

1. THE APPLICATION MUST START FROM THE FILE "INDEX.JS" ON PORT 3000!
2. ONLY THE LAST SUBMISSION WILL BE EVALUATED!

2. Application Overview

Get acquainted with the provided HTML and CSS and create an application for tracking managing space missions.

Visitors can browse the **Home page** and **Dashboard** page with all available Missions. They can also register with an email and password, allowing them to add their own Missions and **like** Missions from other users. Authors can **edit** or **delete** their own Mission entries at any time.

3. Functional Requirements

Guest (Not logged in)

Guest navigation example:



The **application** should provide **Guest** (not logged in) users with the functionality to **Login**, **Register**, and **view** the **Home** page, **Dashboard** page and **Details** page.

Users (Logged in)

User navigation example:



The **application** should provide **Users** (logged in) with the functionality to:

- View the Home page and all other pages with logged-in navigation
- View Dashboard page
- Create a new space mission [Add Mission]
- Access the details page [Details]
- Support missions from other commanders (if the current user is not the owner)
- Delete or Edit their own missions (only for the owner of the current mission)

4. Database Models

The **Database** of the **Space Mission Control** application needs to support **two entities**:

User

- email - string (required),
- password - string (required)

Mission

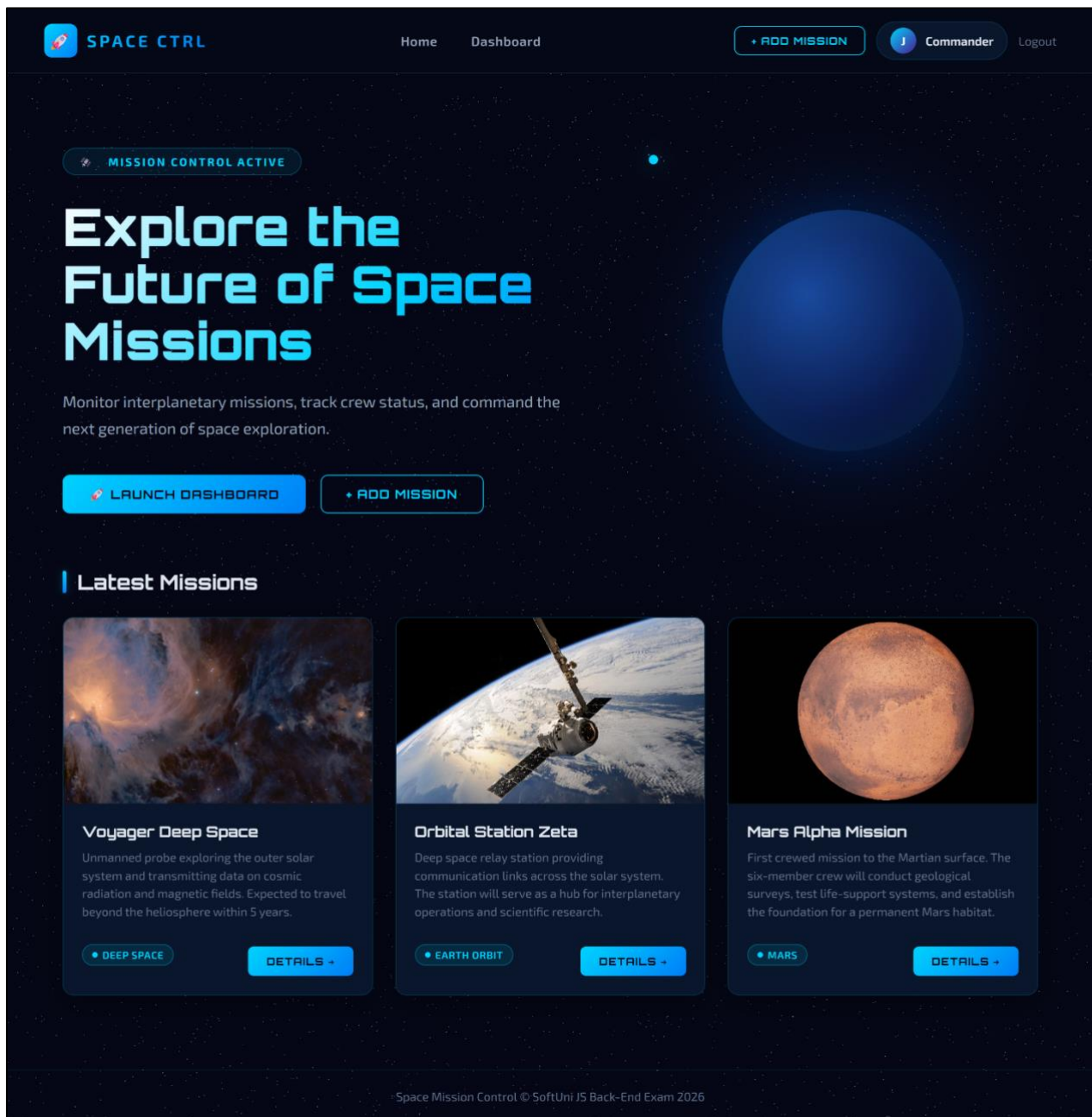
- **name** – String, required (name of the mission)
- **type** – String, required (type of the mission)
- **destination** – String, required (Moon, Mars, Earth Orbit, Jupiter, or Deep Space)
- **imageUrl** – String, required (must start with http:// or https://)
- **crew** – Integer, required (number of crew members, minimum 0)
- **launchDate** – String, required (launch date)
- **duration** – String, required (duration of the mission)
- **description** – String, required (mission objectives and details)
- **createdAt** – **DateTime**, default now() (automatically set on creation)
- **supportList** – collection of Users who supported this mission (relation to User)
- **owner** – relation to the User who created this mission

Note: When a user supports a mission, their id is added to the supportList. You can add more entities if you need.

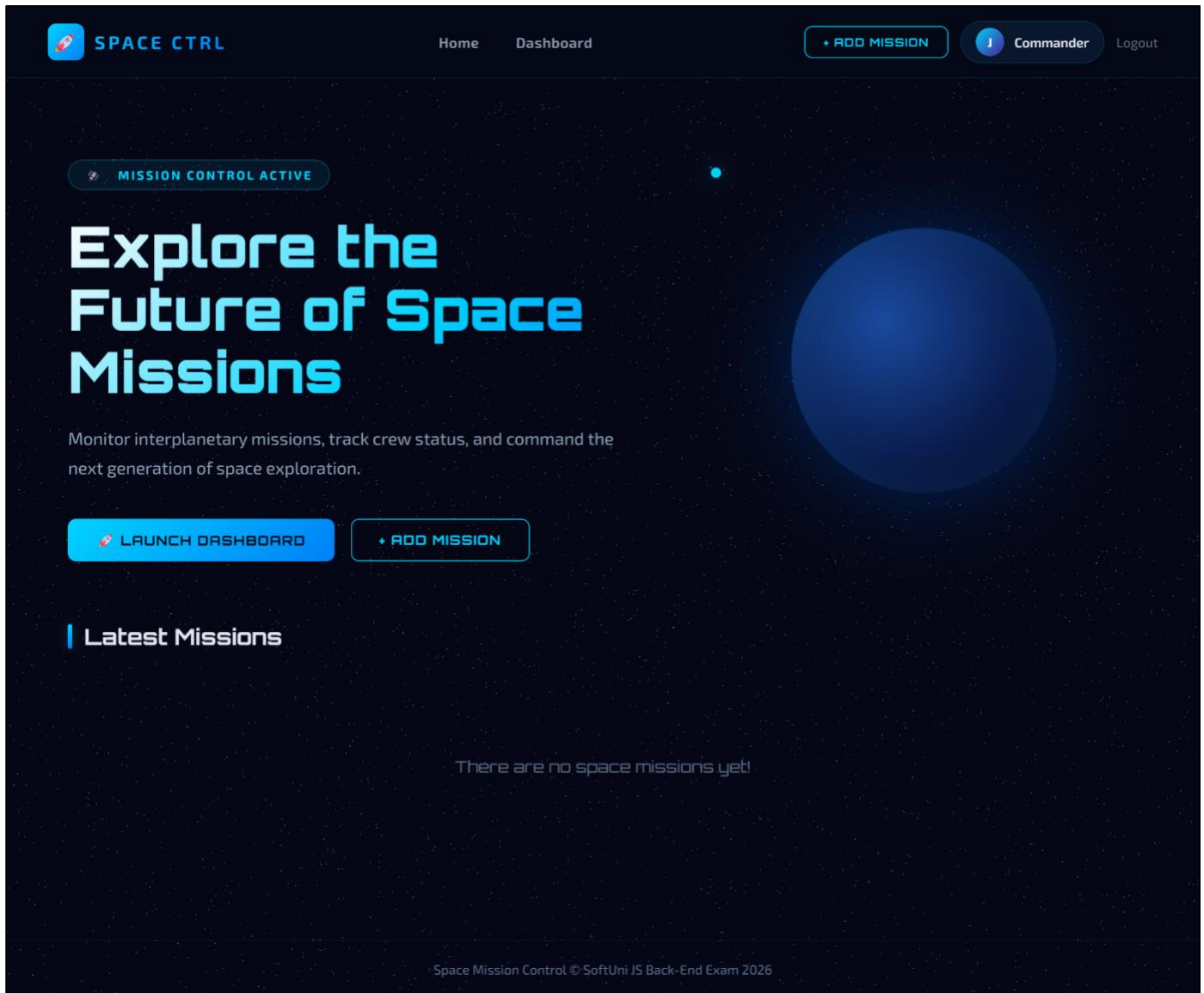
5. Application Pages

Home Page (For logged in users and logged-out users)

Display the **last 3 added missions**. Each mission must show the mission's name, destination, image and a link to the details page.



If there are **NO** missions in the database yet, display **"There are no space missions yet!"**



Register Page (Logged Out User)

Register a user inside the database with an **email and password**. The **password** inside the **database** must be hashed (use **bcrypt**), and both passwords **must match**! After **successful** registration, the user should be logged and **redirected** to the **Home page**. If the user tries to submit **invalid data**, an **error message** should be displayed and the already entered **data should be kept** in the input fields (except for the passwords).

A user who is already logged in should **NOT** be able to reach this .

SPACE CTRL

Home Dashboard LOGIN REGISTER

 **Join the Mission**
Create your Mission Control account

EMAIL ADDRESS

commander@spacectrl.com

PASSWORD CONFIRM PASSWORD

 **CREATE ACCOUNT**

Already have access? [Login here](#)

Space Mission Control © SoftUni JS Back-End Exam 2026

Login Page (Logged Out User)

Logging an already registered user with the correct **email** and **password**. After successful login, you should **redirect the user to the Home page**. If the user tries to submit **invalid data**, an **error message** should be displayed and the already entered **data should be kept** in the input fields (except for the passwords).

A user who is already logged in should **NOT** be able to reach this.

SPACE CTRL

Home Dashboard

LOGIN REGISTER



Mission Login

Access the Space Mission Control Center

EMAIL ADDRESS

commander@spacectrl.com

PASSWORD

 **LAUNCH SESSION**

New to Mission Control? [Request Access](#)

Space Mission Control © SoftUni JS Back-End Exam 2026

Logout (Logged in user)

The logout action is available to **logged-in** users. Upon success, clear any session information and **redirect** the user to the **Home** page.

A user who is **NOT** logged in should **NOT** be able to reach this functionality.

Add Mission (Logged in User)

The **Add Mission** page is available to logged-in users. It contains a form for adding a new space mission. Upon success, redirect the user to the **Dashboard** page. If the user submits invalid data, an error message should be **displayed** and the already entered data should be kept in the input fields.

A guest user (who is not logged in) should **NOT** be able to reach this page.

 Add New Mission

Define the parameters for your new space mission

MISSION NAME

e.g. Mars Alpha Mission

MISSION TYPE

e.g. Crewed Exploration

DESTINATION

Select destination...

IMAGE URL

https://...

CREW MEMBERS

0

LAUNCH DATE

DD/MM/YYYY

DURATION

e.g. 18 months

MISSION DESCRIPTION

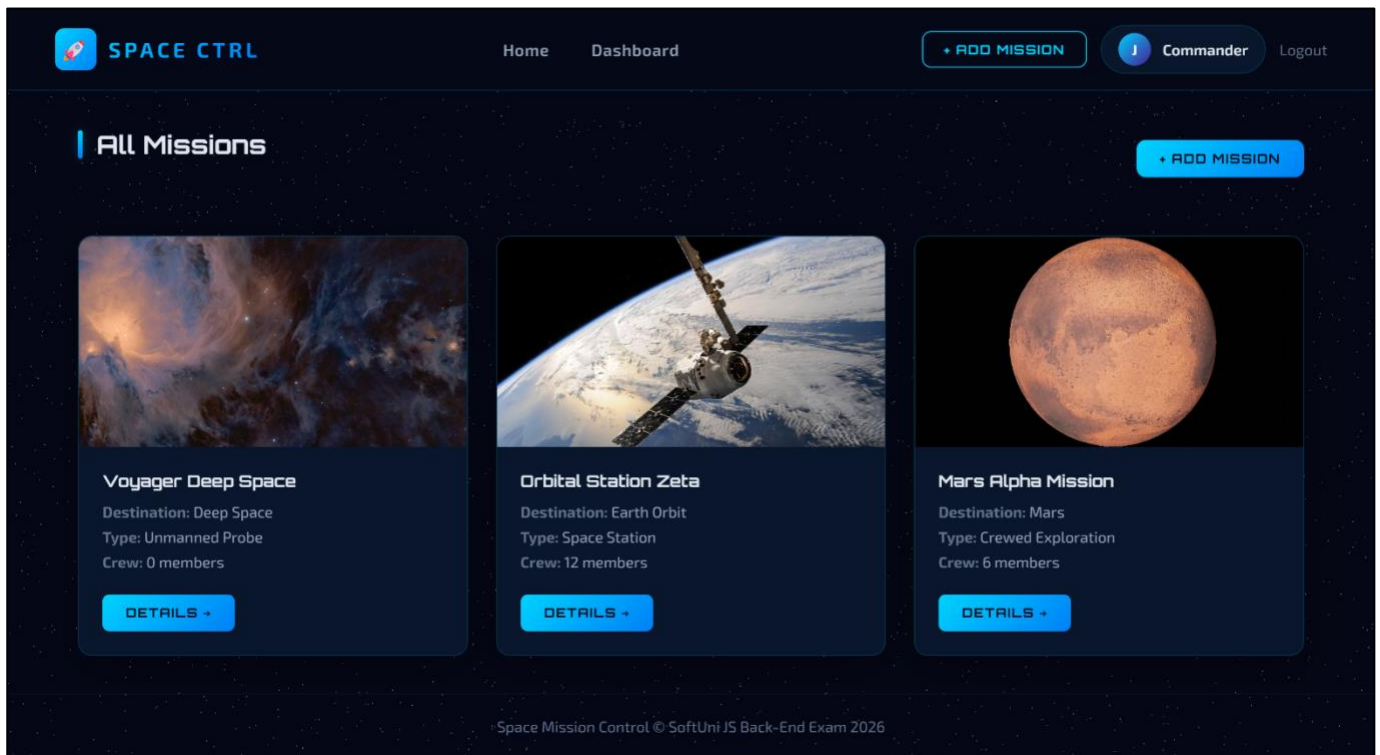
Describe the mission objectives...

 LAUNCH MISSION

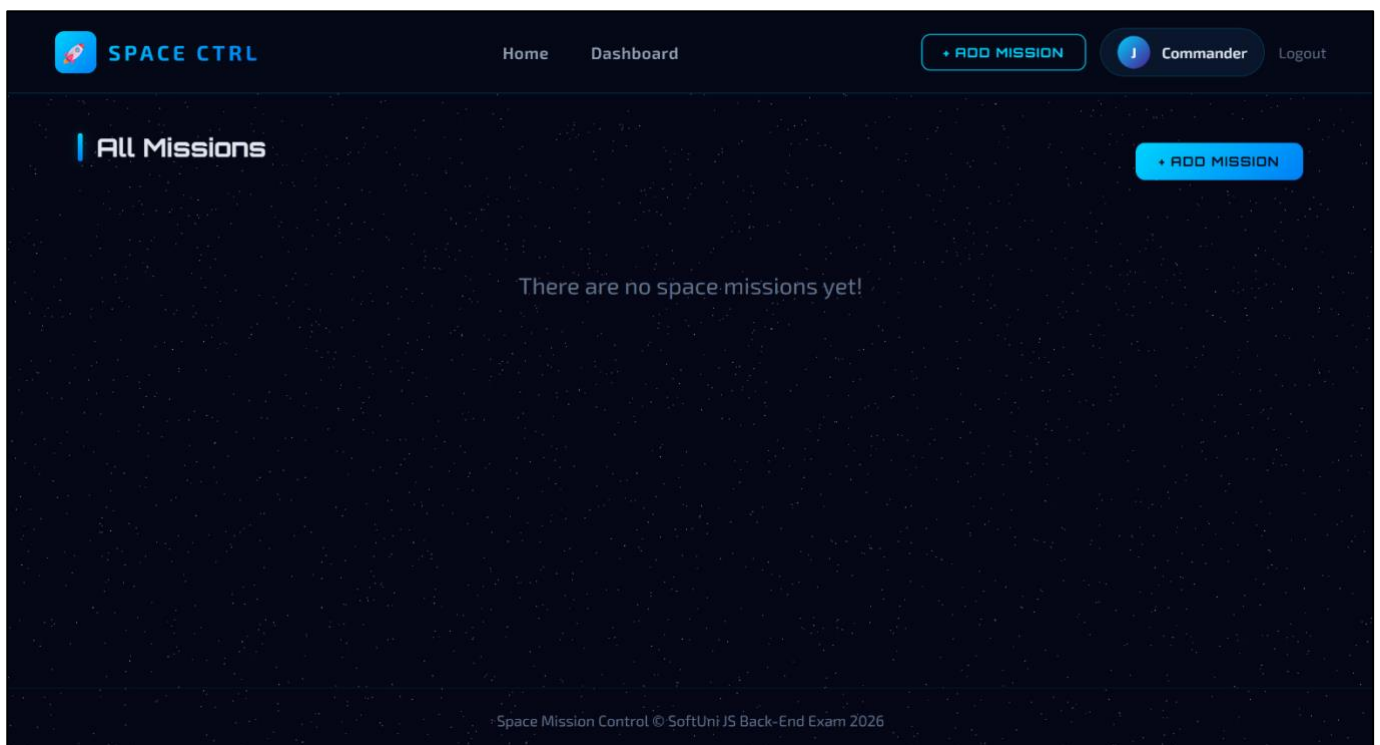
CANCEL

Dashboard (For logged in users and logged out users)

A list of **all missions**. Each mission must be displayed with the **mission image**, **name**, **destination**, **type**, **crew count** and a button for **details**.



The [Details] button should link to the details page for that mission.



If there are **NO** posts, display "There are no space missions yet!"

Details Page (For logged in users and logged out users)

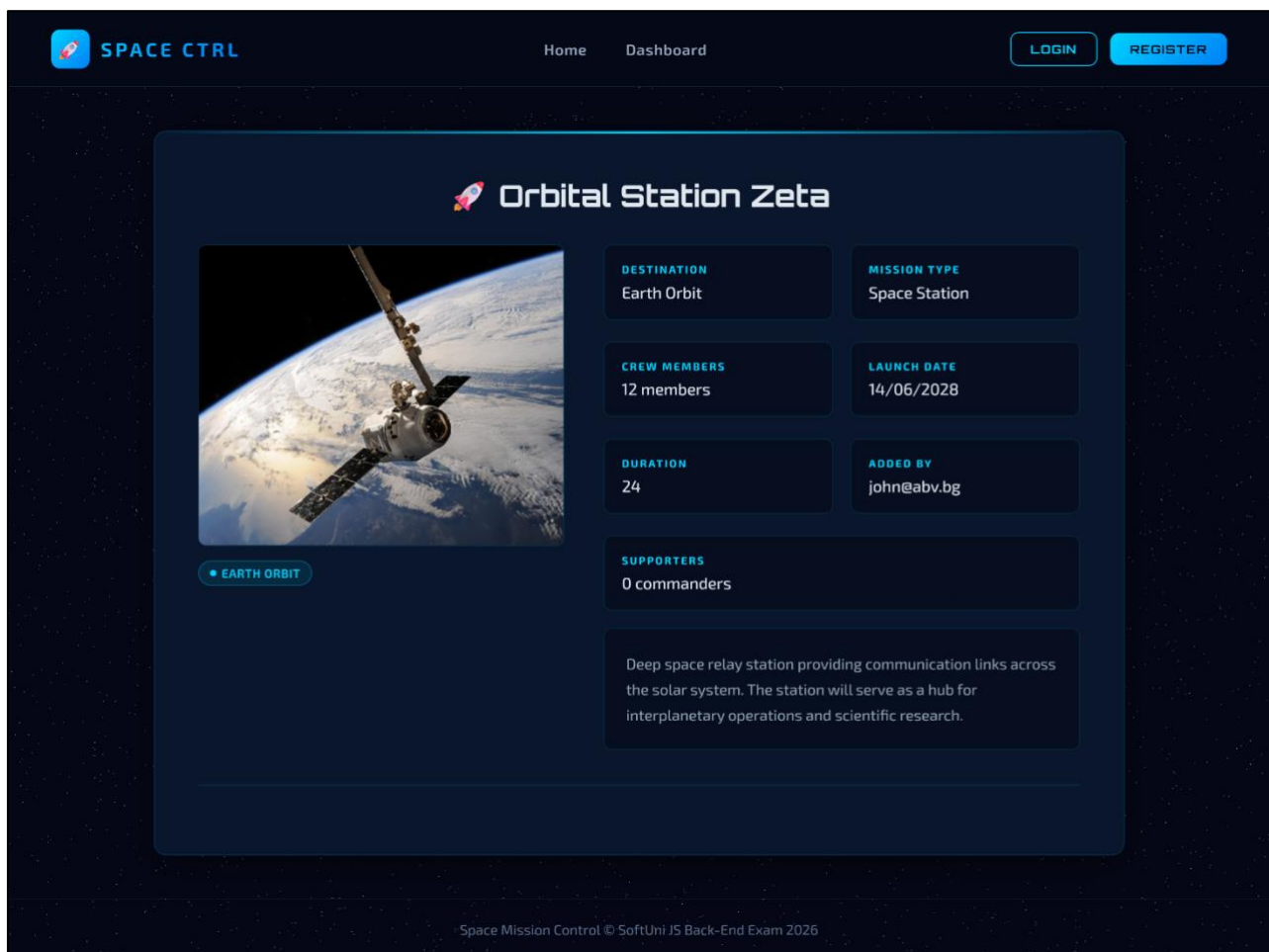
All users should be able to **see the details about the Mission**. Clicking the **Details button** on the post card should display the details page. If the currently registered user **is the creator** of the Mission post, the **Edit** and **Delete** buttons should be displayed. Otherwise, they **should not be available**.

All users should be able to see the details about a mission. The page shows:

- Mission name
- Destination
- Mission type
- Crew members count
- Launch date
- Duration
- Description
- Number of supporters

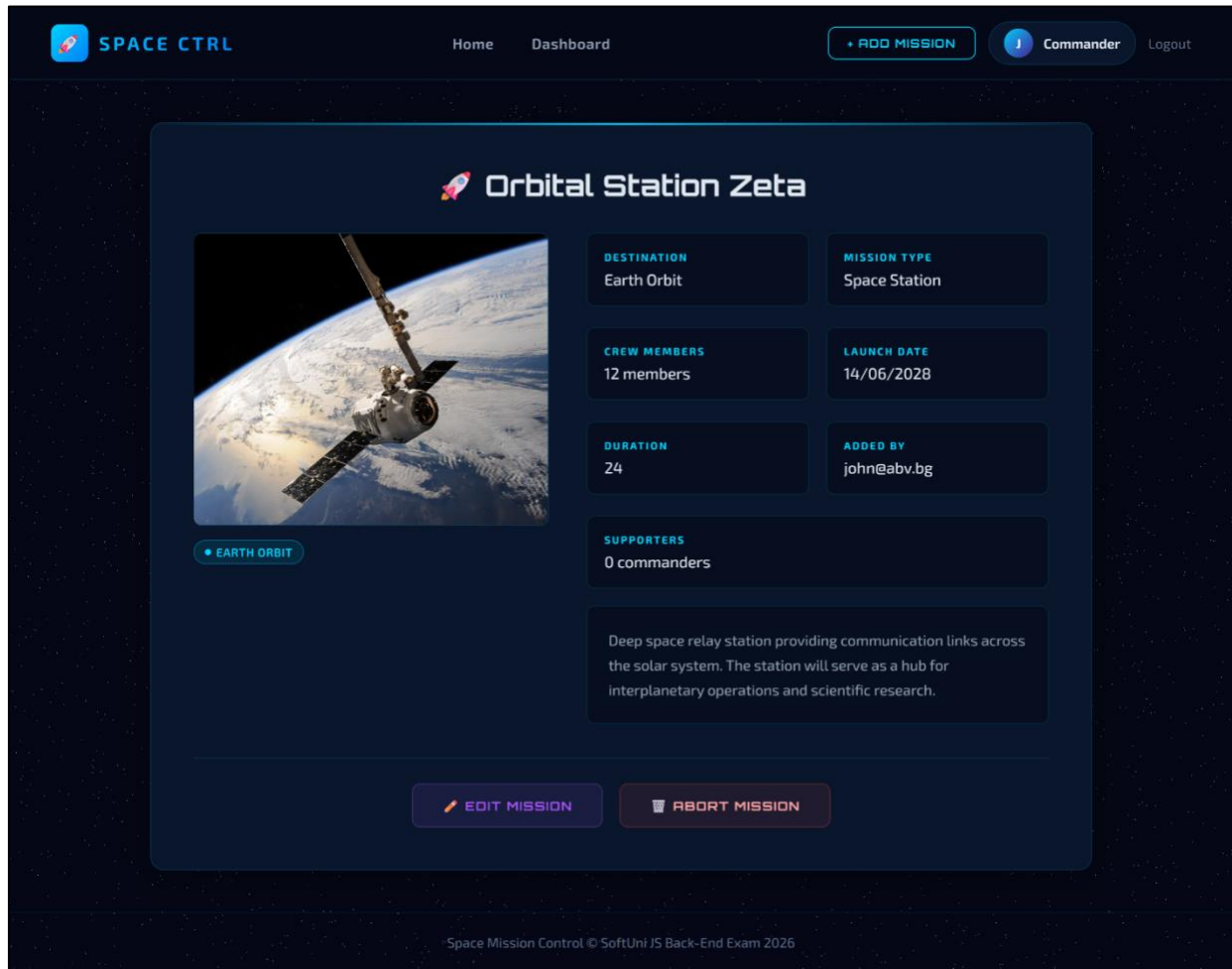
Details Page (Logged out users)

If the user has not **logged in**, **no buttons** should be displayed.



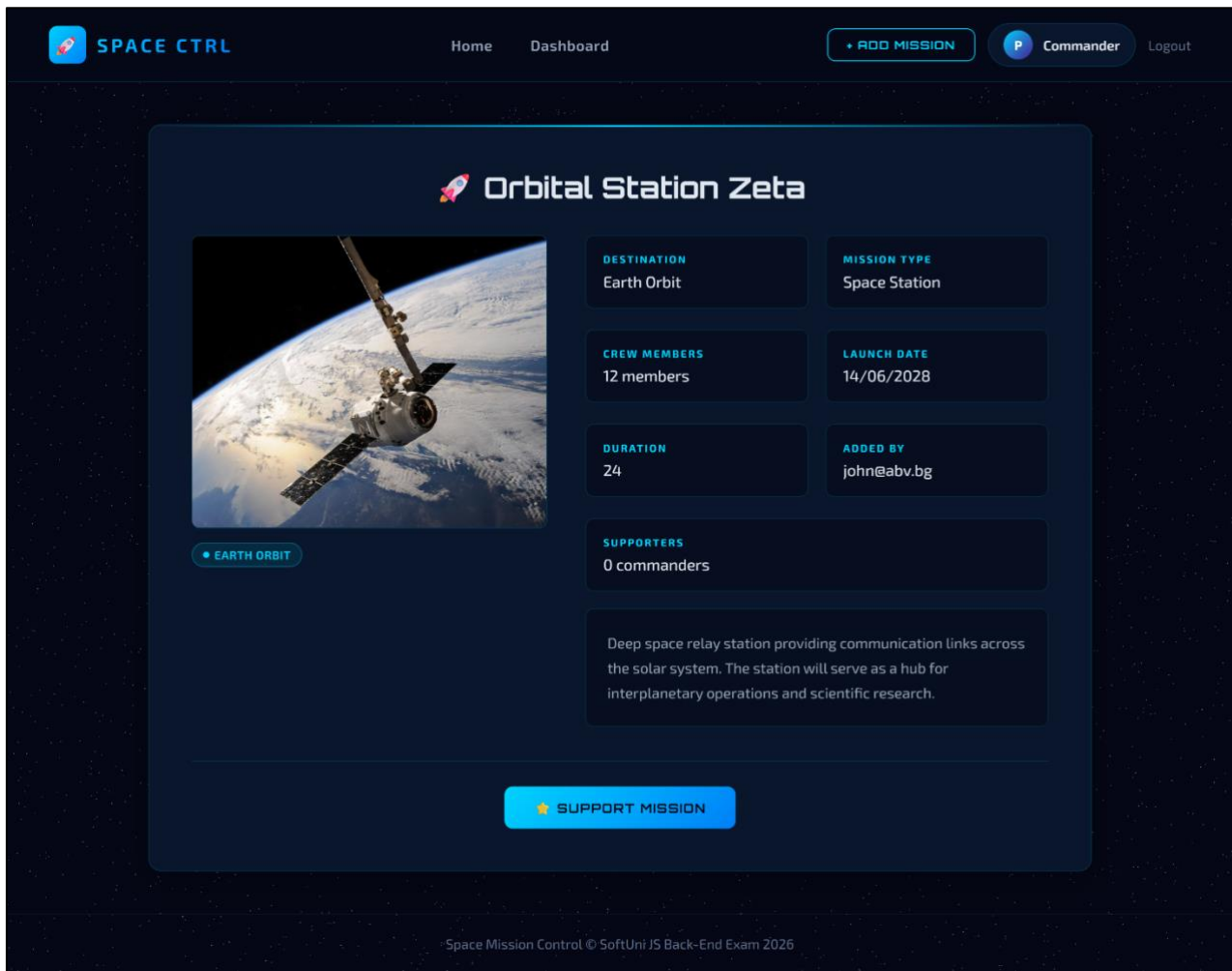
Details Page (Logged in user and creator of the current post)

If the **currently logged-in** user is the **owner** (the user who **created the Mission post**), he should see the **[Abort mission]** and **[Edit mission]** buttons.



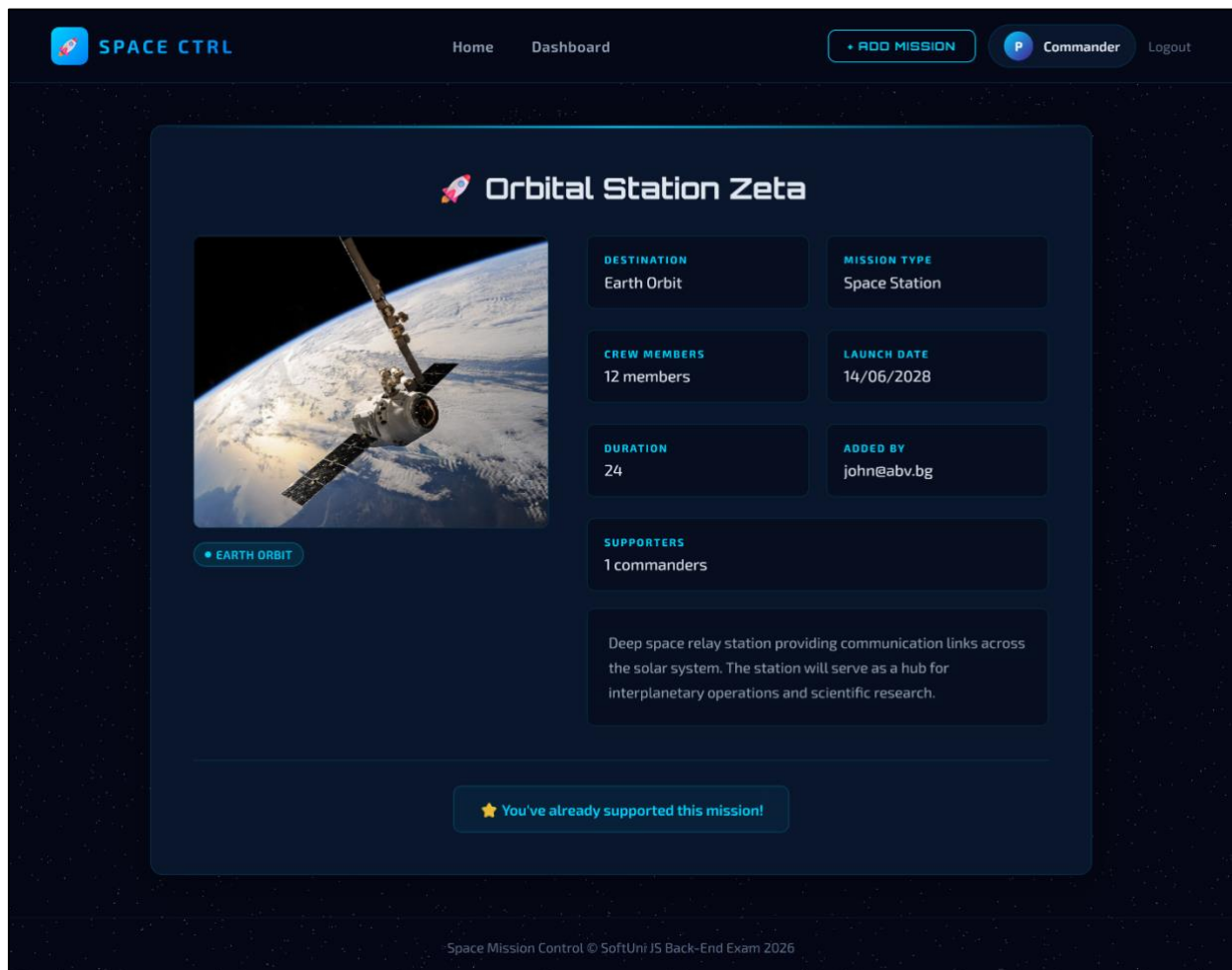
Details Page (Logged in user who has not supported the mission)

If the currently logged-in user is **not the owner** (a user who is **not** the creator of this Mission post) and **he has not supported the post**, he should see a **[Support Mission]** button.



Details Page (Logged in user already liked Mission post)

If the currently logged in user is **not the owner** and has **already liked Mission post**, he should see " **You've already supported this mission!**".



Delete Mission (Logged in user and owner of the current post)

Each owner must be able to delete their mission. After deletion, redirect to the Dashboard page.

A user who is not the owner should NOT be able to reach this functionality.

Edit Mission (Logged in user and owner of the current post)

Each owner can edit their mission. Clicking [**Edit mission**] should display the Edit page with all fields pre-filled with current data. After successful edit, redirect to the details page.

A user who is not the owner should **NOT** be able to reach this page.

SPACE CTRL Home Dashboard [+ ADD MISSION](#) [Commander](#) [Logout](#)

Edit Mission

Update the mission parameters

MISSION NAME Orbital Station Zeta	MISSION TYPE Space Station
DESTINATION Earth Orbit	IMAGE URL https://images.pexels.com/photos/58605/
CREW MEMBERS 12	LAUNCH DATE 14/06/2028
DURATION 24	
MISSION DESCRIPTION Deep space relay station providing communication links across the solar system. The station will serve as a hub for interplanetary operations and scientific research.	

[SAVE CHANGES](#) [CANCEL](#)

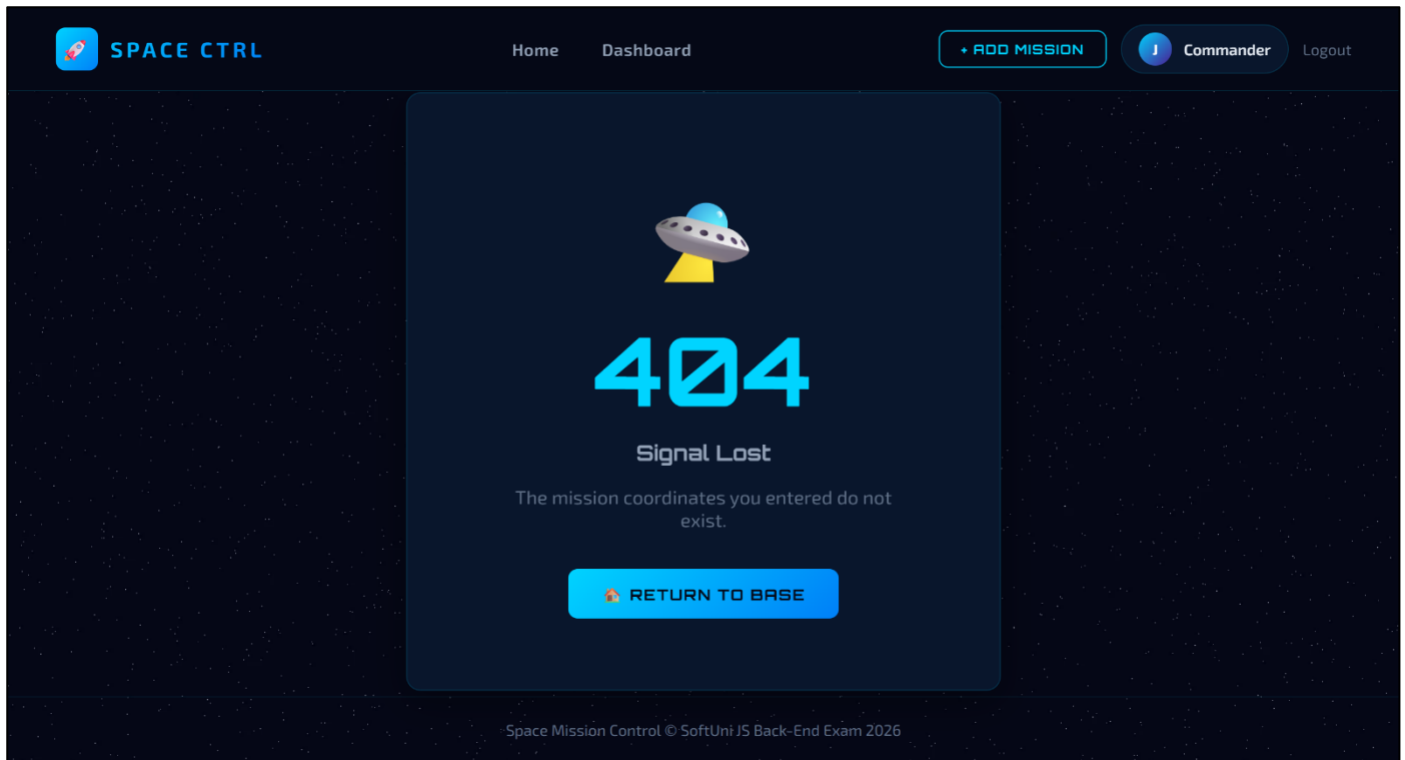
Space Mission Control © SoftUni JS Back-End Exam 2026

6. Security Requirements (Routes Guards)

The **Security Requirements** are mainly **access** requirements. Configurations about which users can access specific functionalities and pages.

- **Guest** (not logged in) users can access the **Home** page.
- **Guest** (not logged in) users can access the **Login** page and functionality.
- **Guest** (not logged in) users can access the **Register** page and functionality.
- **Guest** (not logged in) and **Users** (logged in) can access the **Dashboard (Listed all Mission posts)**.
- **Guest** (not logged in) can access the **Details** page without functionality.
- **Users** (logged in) can access the **Home** page.
- **Users** (logged in) can access the **Details** page and functionality.
 - **Users** (not Mission post owners) can **like other post**.
 - **Users** (Mission post owners) can **Edit** and **Delete** the current Mission post
- **Users** (logged in) can access the **Add Mission** page and functionality.
- **Users** (logged in) can access the **Logout** functionality.

Use the following view for **invalid paths**:



7. Validation and Error Handling

The application should **notify** the users about the result of their actions.

Use Zod for all validations.

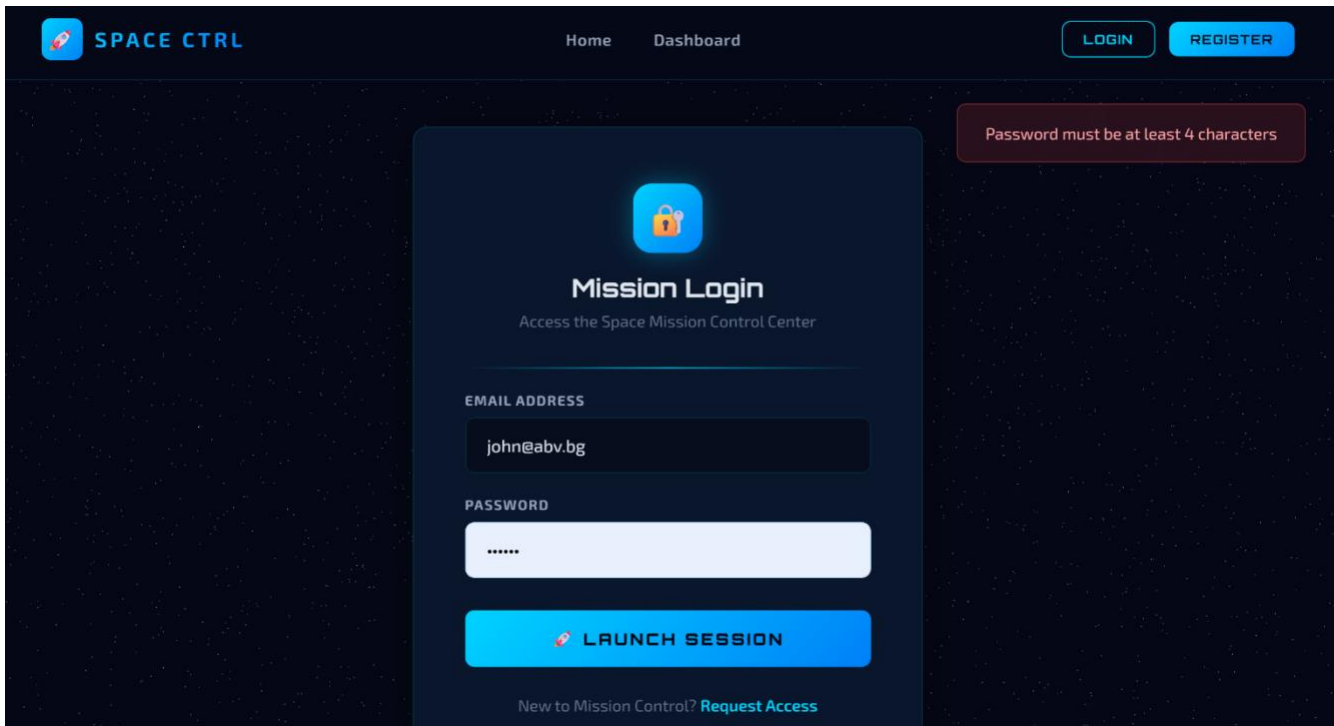
In case of error, you should display div with class "**errorContainer**"

You can choose to display the first error or all of them. You have complete freedom to choose the content of the error message you will display, as long as it is clear what exactly the error is.

Login / Register

You should make the following validations:

- The **email** should be **at least 10 characters** long
- The **password** should be **at least 4 characters** long
- The **repeat password** should be **equal to the password**

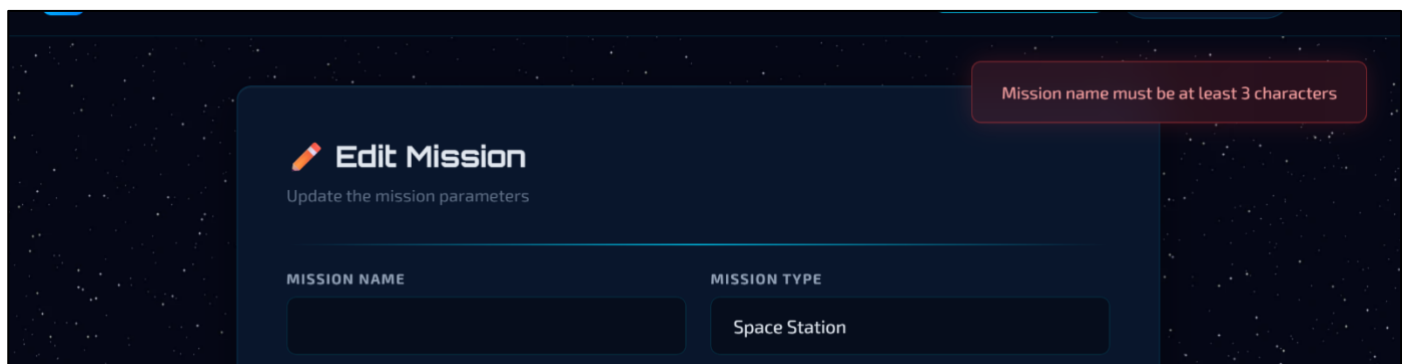


The image shows a 'Mission Login' form on a dark-themed website. The header includes the 'SPACE CTRL' logo, 'Home' and 'Dashboard' links, and 'LOGIN' and 'REGISTER' buttons. The form itself has a blue header with a lock icon and the title 'Mission Login'. Below the title is the subtitle 'Access the Space Mission Control Center'. The form contains two input fields: 'EMAIL ADDRESS' with the value 'john@abv.bg' and 'PASSWORD' with masked characters. A blue 'LAUNCH SESSION' button is positioned below the password field. At the bottom of the form, there is a link 'New to Mission Control? Request Access'. A red error message 'Password must be at least 4 characters' is displayed in the top right corner of the form area.

Missions

You should make the following validations while **creating** or **editing** a **Mission post**:

- **Name** must be at least 3 characters
- **Type** must be at least 5 characters
- **Destination** must be one of: Moon, Mars, Earth Orbit, Jupiter, Deep Space
- **Image URL** must start with `http://` or `https://`
- **Crew** must be a non-negative integer (0 or more)
- **Launch Date** is required
- **Duration** must be at least 2 characters
- **Description** must be at least 10 characters



The image shows an 'Edit Mission' form on a dark-themed website. The header includes the 'Edit Mission' title and the subtitle 'Update the mission parameters'. The form contains two input fields: 'MISSION NAME' and 'MISSION TYPE'. The 'MISSION TYPE' field has the value 'Space Station'. A red error message 'Mission name must be at least 3 characters' is displayed in the top right corner of the form area.

8. REST API Endpoint

Objective

Implement a REST API endpoint that allows external clients to retrieve the latest missions in JSON format. The endpoint can be tested using Postman or any other REST client.

Endpoint Specification

You can make request to the following endpoint:

- **Latest Missions (GET):** <http://localhost:3000/api/missions/latest>

Returns the **3 most recently added missions**, sorted by **createdAt** in descending order.

Each mission must include:

- name
- type
- destination
- imageUrl
- crew
- launchDate
- duration
- description
- ownerEmail
- createdAt

Access Requirements

- Endpoint must be **publicly accessible**.
- No authentication required.

Testing with Postman

- Open Postman and create a new **GET** request
- Enter the URL: **<http://localhost:3000/api/missions/latest>**
- Click Send — you should receive a JSON array with up to 3 missions
- Verify that the response contains all required fields listed above

http://localhost:3000/api/missions/latest

GET http://localhost:3000/api/missions/latest

Params Authorization Headers (6) Body Pre-request Script Tests Settings

Query Params

Key	Value
Key	Value

Body Cookies Headers (7) Test Results

Status: 200 OK Time: 19 ms Size: 1.61 KB Save Response

Pretty Raw Preview Visualize JSON

```

1 {
2   {
3     "name": "Voyager Deep Space",
4     "type": "Unmanned Probe",
5     "destination": "Deep Space",
6     "imageUrl": "https://images.pexels.com/photos/27295206/pexels-photo-27295206.jpeg",
7     "crew": 0,
8     "launchDate": "26/09/2038",
9     "duration": "68",
10    "description": "Unmanned probe exploring the outer solar system and transmitting data on cosmic radiation and magnetic fields. Expected to travel beyond the heliosphere within 8 years.",
11    "ownerEmail": "john@bvb.bg",
12    "createdAt": "2026-07-29T16:18:16.918Z"
13  },
14  {
15    "name": "Orbital Station Zeta",
16    "type": "Space Station",
17    "destination": "Earth Orbit",
18    "imageUrl": "https://images.pexels.com/photos/586866/pexels-photo-586866.jpeg",
19    "crew": 12,
20    "launchDate": "14/06/2028",
21    "duration": "24",
22    "description": "Deep space relay station providing communication links across the solar system. The station will serve as a hub for interplanetary operations and scientific research.",
23    "ownerEmail": "john@bvb.bg",
24    "createdAt": "2026-07-29T16:09:08.389Z"
25  },
26  {
27    "name": "Mars Alpha Mission",
28    "type": "Crewed Exploration",
29    "destination": "Mars",
30    "imageUrl": "https://images.pexels.com/photos/28337611/pexels-photo-28337611.jpeg",
31    "crew": 6,
32    "launchDate": "15/03/2032",
33    "duration": "18",
34    "description": "First crewed mission to the Martian surface. The six-member crew will conduct geological surveys, test life-support systems, and establish the foundation for a permanent Mars habitat.",
35    "ownerEmail": "john@bvb.bg",
36    "createdAt": "2026-07-29T16:07:48.433Z"
37  }
38 }

```

GOOD LUCK! 😊